Checkpoint 3: An Order-Controlled Search Agent for Misstatement-Risk Screening

Ryan Khan IDAI-780, Fall 2026 Instructor: Zhiqiang Tao Code: github.com/<user>/fraud-cot-search-agent

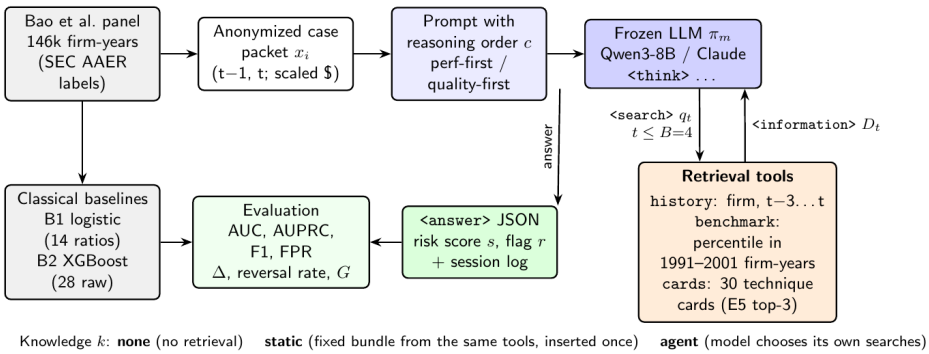


Figure 1: Framework. A frozen LLM reasons in order c and, in the agent setting, issues up to B=4 searches to three leakage-safe tools before answering; outputs are scored with the same metrics as the classical baselines.

1 Formulation

Data and task. We use the Bao et al. [1] panel of U.S. firm-years with SEC AAER labels $y_i \in \{0, 1\}$. Splits are temporal: train 1991–2001, test 2003–2008, with a two-year gap and serial-fraud relabelling. Case i is an anonymized packet x_i with 28 items for years $t-1, t$ and 14 ratios. Firm ID and calendar years are removed, and dollar amounts are multiplied by a random constant $\lambda_i \in [0.5, 2]$.

Inference model. A frozen LLM π_m ($m \in$ {Qwen3-8B, Claude Sonnet 5.5}) runs under reasoning order $c \in \{\mathrm{D}, \mathrm{P}, \mathrm{Q}\}$: direct; *performance-first* (growth, margins, ROA, the headline story); and *quality-first* (accruals, cash, receivables, inventory, soft assets). The two CoT prompts contain the same words in a different order. Knowledge is $k \in \{\varnothing, \mathrm{S}, \mathrm{A}\}$: none, static bundle, or search agent. Following Search-R1 [2], without RL, the agent produces

$$\pi_m(\tau \mid x_i; c) = \prod_{t=1}^{T} \pi_m(t_t, q_t \mid x_i, c, \tau_{<t}), \quad T \le B, \tag{1}$$

where t_t is a `<think>` step and q_t a `<search>` call to one of three tools. `history` returns the firm's own $t-3..t$ values scaled by λ_i. `benchmark` returns the percentile of a ratio among 1991–2001 firm-years. `cards` returns the top-3 of 30 technique cards,

$$D_t = \mathrm{TopK}_{d \in \mathcal{T}} \cos\big(\phi(q_t), \phi(d)\big), \tag{2}$$

where ϕ is the E5 encoder. Each D_t is inserted inside `<information>`; the model conditions on it but does not generate it. The answer gives a risk score $s_i^{m,c,k} \in [0, 100]$ and a red flag $r_i = \mathbb{1}[s_i \ge \theta_{m,c,k}]$, with θ chosen to maximize F1 on a disjoint dev split (2000–01 cases).

Hypotheses. *H1 (order bias):* $\Delta_i^{m,k} = s_i^{m,\mathrm{P},k} - s_i^{m,\mathrm{Q},k}$ is negative, i.e. reasoning that starts from performance anchors on the headline numbers. We report mean Δ, $|\Delta|$, the red-flag reversal rate ρ, and a Wilcoxon signed-rank test. *H3 (knowledge):* $G^{m,c} = \mathrm{AUPRC(A)} - \mathrm{AUPRC(S)} > 0$, and $|\Delta^{\mathrm{A}}| < |\Delta^{\varnothing}|$. The agent's own queries can show the mechanism: we measure the share of lookups aimed at quality items under each order.

2 Comparison to Baselines

Model	Condition (c, k)	AUC	AUPRC	$\lvert\Delta\rvert$
B0	random	0.58	0.57	–
B1	logistic, 14 ratios	0.62	0.59	–
B2	XGBoost, 28 items	**0.76**	**0.76**	–
Qwen3-8B	D, ∅ (CP2 Direct)	–	–	–
	P / Q, ∅ (CoT)	–	–	–
	P / Q, S (static RAG)	–	–	–
	P / Q, A (agent)	–	–	–
Claude	same grid	–	–	–

Table 1: The same 60 balanced out-of-sample test cases (30 later-AAER). Values have 95% case-bootstrap CIs (in repo). On the full test set (35,166 firm-years) B2 reaches AUC 0.725 and AUPRC 0.015 against a prevalence of 0.0074, in line with Bao et al.'s RUSBoost (≈0.71).

Algorithm 1: Order-controlled search agent (batched)

Input: cases $\{x_i\}$, model π_m, order c, budget B=4
$h_i \leftarrow \textsc{Prompt}(x_i, c)$ for all i; active $\leftarrow$ all;
while *active* $\neq \varnothing$ *and step* $< B$+4 **do**
  $y_i \leftarrow \pi_m(h_i)$ until `</search>` | `</answer>`; // batched
  foreach *active* i **do**
    if `<answer>` *in* y_i **then**
      close i
    else if $(tool, q) \leftarrow \textsc{Parse}(y_i)$, $n_i < B$ **then**
      $D \leftarrow \textsc{Tool}(tool, q)$;
      h_i += y_i‖`<information>`D`</information>`; log $(q, \mathrm{IDs}(D))$; n_i++
    else "rethink" once (or budget used): force `<answer>`;
  end
end
return $\textsc{ParseJSON}(h_i)$ for all i;

Qualitative. Classical models see only numbers, so order cannot affect them. Static RAG gives every case the same bundle. The agent chooses what to check and every check is logged, so we can ask *why* order matters: does performance-first reasoning lead to fewer quality checks?

3 Implementation Details

Training (B2). XGBoost minimizes $\sum_i w_{y_i} \ell(y_i, f(z_i)) + \Omega(f)$ with $w_1 = n_0/n_1$. Hyperparameters are chosen on fit 1991–97 → validation 2000–01 over depth $\{3, 4, 6\}\times$ trees $\{200, 400\}$ (learning rate 0.05, row and column subsample 0.8). The selected setting (depth 3, 200 trees) is refit on 1991–2001 with median imputation and 1/99% winsorizing fit on training rows only. We log per-round log loss, AUC and AUPRC, save checkpoints every 50 rounds, and score the test years once. **LLM testing.** Qwen3-8B runs in bf16 with Transformers on one A100, with greedy decoding, built-in thinking off, at most 1,024 new tokens per step, and batched lock-step rollouts (16 sessions per batch). Claude runs the same tools through native API tool use at effort low (D) or high (CoT), since it does not write out its reasoning. Static RAG covers 4 history items, 13 ratio benchmarks and the top-6 cards. Every session logs the case, condition, prompt hash, queries, retrieved IDs, malformed or forced flags and raw output. There are 13 unit tests covering the parser, budget cutoff, rethink path, leakage guards, retrieval determinism, prompt symmetry and schema. Uncertainty comes from 1,000 case-bootstrap resamples, paired across conditions.

References

[1] Y. Bao et al. Detecting accounting fraud in publicly traded U.S. firms using a machine learning approach. *J. Accounting Research*, 58(1), 2020.

[2] B. Jin et al. Search-R1: Training LLMs to reason and leverage search engines with reinforcement learning. *COLM*, 2025.